

Chapter 05

Structured Output in LangChain

Turning free-form LLM text into well-defined, program-friendly data — using TypedDict, Pydantic, and JSON Schema with `with_structured_output()`.

WHAT THIS CHAPTER COVERS

- ▶ Structured vs. unstructured output
- ▶ Why structured output matters
- ▶ Three real-world use cases
- ▶ The `with_structured_output()` function
- ▶ Defining schemas with TypedDict
- ▶ Defining schemas with Pydantic
- ▶ Defining schemas with JSON Schema
- ▶ Choosing the right approach & model limitations

Structured Output in LangChain

RECAP — CHAPTER 4

Chapter 4 covered Prompts — how to construct flexible, reusable inputs that we send to an LLM. That chapter was about controlling what goes **into** the model. This chapter flips the focus to what comes **out**: how to make an LLM's response arrive in a predictable, machine-readable shape instead of free-flowing text.

1. What Is Structured Output?

Definition

When you talk to an LLM normally — for example through ChatGPT — you send a text prompt and get back a text response. Since the response is plain text with no fixed shape, this is called *unstructured output*. **Structured output**, by contrast, refers to the practice of having a language model return its response in a well-defined data format (such as JSON) instead of free-form prose.

Why It Exists

A response in plain English is easy for a human to read but hard for a program to reliably parse. If the same information is returned as JSON with fixed keys, any downstream code can extract fields programmatically without fragile string-parsing logic. This is the single biggest benefit of structured output: it lets an LLM be integrated with other systems — databases, APIs, and tools — instead of only talking to humans.

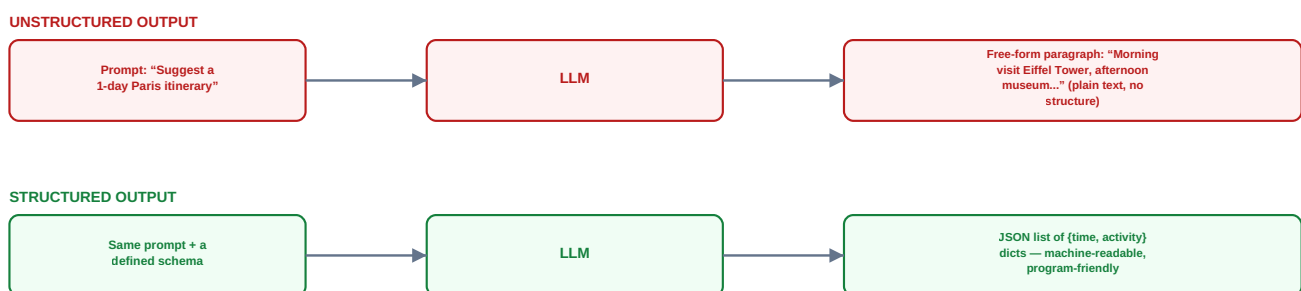


Figure 5.1 — Unstructured free-text response vs. structured JSON response for the same prompt

Example

Prompt: "Can you create a one-day travel itinerary for Paris?"

Unstructured response: "Here is a suggested itinerary. Morning: visit the Eiffel Tower. Afternoon: visit a museum. Evening: have dinner."

Structured response: a JSON array of three dictionaries, each with a `time` key and an `activity` key — directly loopable and storable in code.

IMPORTANT

Structured output does not mean the LLM "understands" data differently — it simply means the response is shaped according to a schema you define, so your program can consume it reliably.

🔄 Quick Revision — What Is Structured Output

- Unstructured output = plain text response with no fixed shape.
- Structured output = response returned in a well-defined data format (typically JSON).
- Structured output makes model output easy to parse and work with programmatically.
- The core benefit: LLMs can now integrate with other systems, not just humans.

2. Why Do We Need Structured Output?

Problem It Solves

Historically, LLMs were used purely for human-to-model conversation: a human sends a message, the model replies with text, and both sides understand each other. The problem is that this textual, unstructured output cannot be reliably connected to other systems such as databases or APIs. Structured output solves this by giving the LLM's response a defined data format, which finally allows the LLM to talk to *machines* as well as humans.

How It Works — Three Real-World Use Cases

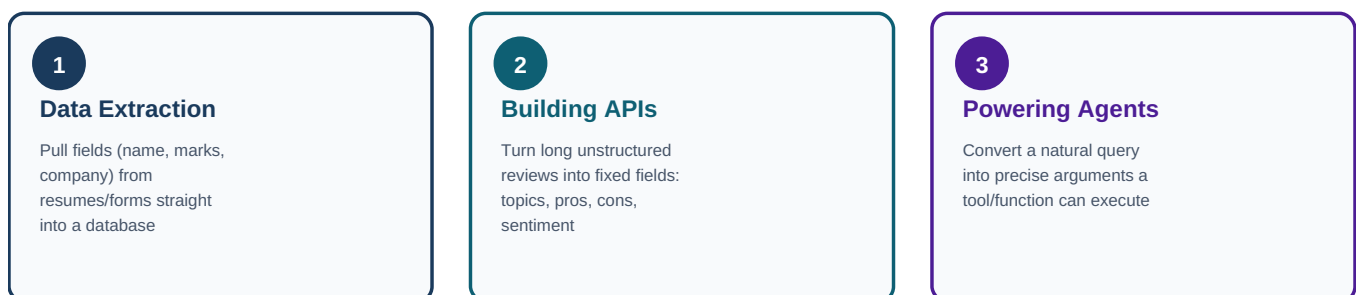


Figure 5.2 — Three practical use cases for structured output

1. DATA EXTRACTION

Example: a job portal like Naukri.com where candidates upload resumes. Instead of storing raw resume text, the system extracts specific fields — name, last company, 10th/12th marks, college marks — and inserts them straight into a database. The resume text is sent to an LLM, the LLM returns JSON with these fields, and code then inserts that JSON into the database.

2. BUILDING APIS

Example: a review-analysis API for a site like Amazon. Reviews are long and unstructured. A structured-output pipeline can extract the topics discussed (battery, display, processor), collect all pros in one place, all cons in another, and compute an overall sentiment — then expose all of this through a Flask/FastAPI endpoint that anyone can consume.

3. POWERING AGENTS

Agents rely on tools (e.g. a calculator tool) to perform actions. A tool like a calculator expects numbers, not a sentence like "Find the square root of two." Running structured output on the incoming query extracts the exact numeric arguments the tool needs, so the agent can call the tool correctly.

REAL WORLD INSIGHT

These three use cases are the most common, but they are not exhaustive — structured output is a general-purpose technique useful anywhere an LLM's response needs to feed into further logic.

Quick Revision — Why Structured Output

- Without structure, LLMs could only talk to humans, not other systems.
- Use case 1: Data Extraction — e.g. pulling resume fields into a database.
- Use case 2: Building APIs — e.g. extracting topics/pros/cons/sentiment from reviews.
- Use case 3: Powering Agents — converting a query into precise tool arguments.
- Structured output is the bridge between LLMs and machine-to-machine communication.

3. Two Ways to Get Structured Output

How It Works

Not all LLMs behave the same way when it comes to producing structured data:

Model Category	Behaviour	LangChain Mechanism
Models trained to natively generate structured output (e.g. OpenAI GPT models)	Can directly return JSON-formatted responses when instructed	<code>with_structured_output()</code> function
Models without native structured-output capability	Only return free-form text; can still be forced into structure	Output Parsers (covered in the next chapter)

IMPORTANT

This chapter focuses only on models that can natively produce structured output, using the `with_structured_output()` function. Output Parsers — the technique for models that cannot — are covered separately.

Quick Revision — Two Ways

- Some LLMs (e.g. OpenAI GPT models) natively support structured output.
- For these, LangChain provides the `with_structured_output()` function.
- Other LLMs lack this capability entirely and need Output Parsers instead.
- Output Parsers are LangChain classes that impose structure on any LLM's raw text output.

4. The `with_structured_output()` Function

Definition

`with_structured_output()` is a method available on chat models in LangChain. It lets you specify the exact data format you want back, and the model returns its response in that format. The only change to a normal invocation flow is that you call this function — passing in your schema — before invoking the model.

How It Works

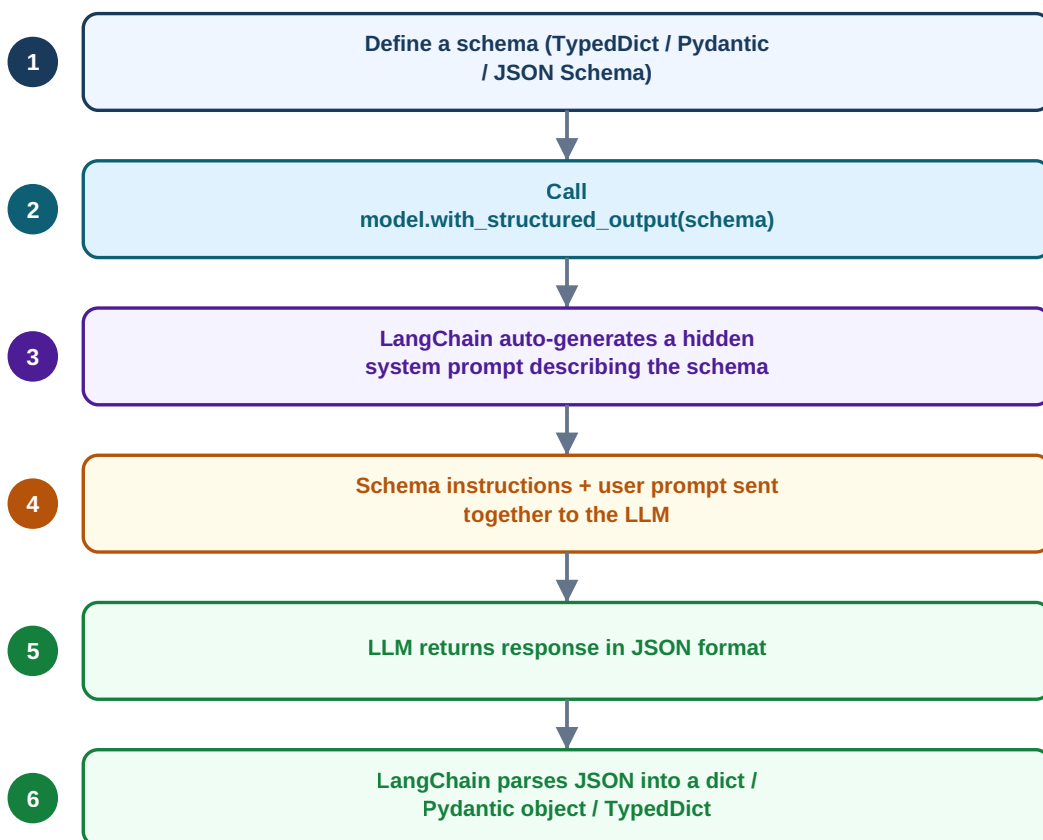


Figure 5.3 — What happens internally when `with_structured_output()` is called

Behind the scenes, LangChain generates a hidden system prompt describing the schema (its fields and their descriptions), attaches it ahead of the user's actual prompt, and sends the combined prompt to the LLM. Because the underlying model

is trained to return JSON, it responds in JSON — which LangChain then parses back into a Python dict, a Pydantic object, or a TypedDict-shaped dict, depending on the schema type used.

Three Ways to Define a Schema

A schema can be specified in three different ways, each suited to different needs: **TypedDict**, **Pydantic**, and **JSON Schema**.

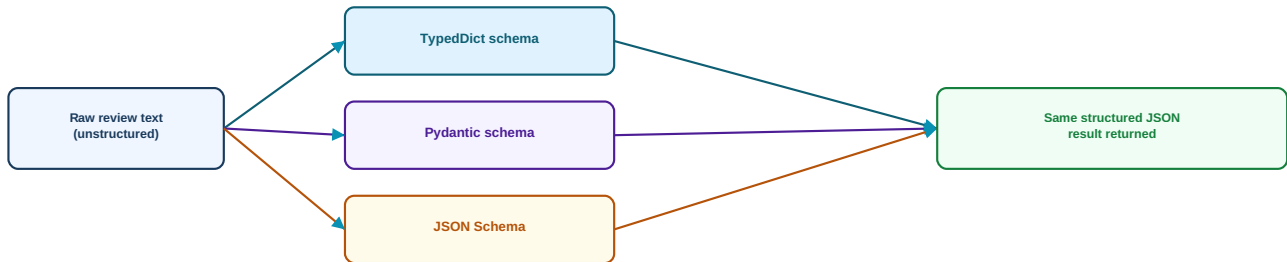


Figure 5.4 — Same input, three schema-definition approaches, same structured JSON output

Quick Revision — `with_structured_output()`

- `with_structured_output(schema)` is called before invoking the model.
- It internally builds a hidden system prompt describing the schema.
- The model returns JSON because it is trained to do so once instructed.
- LangChain parses the JSON back into dict / Pydantic object / TypedDict.
- Three schema formats are supported: TypedDict, Pydantic, JSON Schema.

5. Schema Approach 1 — TypedDict

Definition

`TypedDict` is a way to define a dictionary in Python where you specify what keys and value types should exist. It helps ensure a dictionary follows a specific structure — but purely as a hint for editors/type-checkers, not as enforced validation.

Why It Exists

Normally in Python, dictionaries are built ad-hoc: nothing stops one collaborator from putting a string where another expects an integer, which can cause runtime bugs. `TypedDict` solves this by letting you first declare, as a class, exactly what keys a dictionary should have and what type each value should be.

```

from typing import TypedDict

class Person(TypedDict):
    name : str
    age : int

new_person : Person = {
    "name" : "Saurabh",
    "age" : 20
}

# This still runs without error even though age is a string --
# TypedDict gives editor hints only, it does NOT validate at runtime
new_person2 : Person = {
    "name" : "Saurabh",
    "age" : '20'
}
print(new_person)
print(new_person2)

```

Both dictionaries print successfully — the type editor will flag '20' as a discrepancy, but nothing stops the code from running with the wrong type.

COMMON MISTAKE

Assuming TypedDict enforces types at runtime. It does not — it only tells your editor/type-checker what the shape should be. There is no actual validation or error raised if the wrong type is supplied.

Using TypedDict with with_structured_output()

The uploaded implementation uses `HuggingFaceEndpoint` with `ChatHuggingFace` and the `Qwen/Qwen2.5-7B-Instruct` model rather than `OpenAI`, since HuggingFace-hosted chat models don't accept Pydantic schemas directly — TypedDict works fine with them.

```

from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from typing import TypedDict, Annotated, Optional, Literal
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(
    repo_id = "Qwen/Qwen2.5-7B-Instruct"
)
model = ChatHuggingFace(llm = llm)

# Schema: each key is annotated with a description string,
# which LangChain feeds to the model as guidance
class Review(TypedDict):
    key_themes : Annotated[list[str], "Identify key themes discussed and write them down in list."]
    summary : Annotated[str, "Generate a brief summary of the review."]
    sentiment : Annotated[Literal["pos", "neg"], "Determine the sentiment of the review."]
    pros : Annotated[Optional[list[str]], "List down all the pros of the product in a list."]
    cons : Annotated[Optional[list[str]], "List down all the cons of the product in a list."]
    name : Annotated[Optional[str], "Identify the name of the reviewer. Don't make up a name if it is not mentioned."]

# Key step: bind the schema to the model BEFORE invoking
structured_model = model.with_structured_output(Review)

result = structured_model.invoke("""...long product review text...""")
print(result)

```

This returns a regular Python dictionary whose keys match the schema. Fields can be fetched with normal dict syntax, e.g. `result["summary"]` and `result["sentiment"]`.

Key Building Blocks

Feature	Purpose
<code>Annotated[type, "description"]</code>	Attaches a natural-language description to a field so the model understands exactly what to produce, reducing ambiguity for single-word field names.
<code>Literal["pos", "neg"]</code>	Restricts a field to a fixed set of allowed values instead of any string.
<code>Optional[...]</code>	Marks a field as not guaranteed to appear (e.g. a reviewer's name may not be mentioned).

BEST PRACTICE

Always add an `Annotated` description to every field. It guides the LLM's interpretation even when a field name alone might be ambiguous, and it costs nothing extra to add.

Quick Revision — TypedDict

- TypedDict defines the expected keys and value types of a dictionary.
- It provides editor/type hints only — no runtime validation, no default values.
- Annotated[] attaches descriptions that guide the LLM when used inside a schema.
- Literal[] restricts a field to a fixed set of values (e.g. "pos"/"neg").
- Optional[] marks a field as not guaranteed to be present in the response.
- Best suited for pure-Python projects where no cross-language schema sharing is needed.

6. Schema Approach 2 — Pydantic

Definition

Pydantic is a data validation and data parsing library for Python. Unlike TypedDict, it actually enforces the constraints you declare, raising errors when data doesn't match — which is why frameworks like FastAPI rely on it heavily.

Why It Exists

When data flows into a system — from an API caller, or from an LLM — you often need certainty that the data satisfies specific criteria before your program proceeds. TypedDict cannot guarantee this; Pydantic can, by validating and (where possible) auto-converting data at object-creation time.

Basic Example

```
from pydantic import BaseModel, EmailStr, Field
from typing import Optional

class Student(BaseModel):
    name : str
    age : Optional[int] = None
    email : EmailStr
    cgpa : float = Field(gt = 0, lt = 10, default = 5,
                        description = "This float number represents the cgpa of a student")

# This will raise a validation error if a wrong data type is passed for "name"
new_student = {"name" : "Saurabh", "age" : 20, "email" : "abc@gmail.com", "cgpa" : 9.32}
student = Student(**new_student)

student_dict = dict(student)
print(student_dict["age"])
```

If name were given an integer instead of a string, Pydantic raises "Input should be a valid string" and object creation fails — a validation guarantee TypedDict cannot offer.

Pydantic Feature Set

1. DEFAULT VALUES

A field can be given a plain default (e.g. `name: str = "Nitish"`), so if no value is supplied, the default is used automatically.

2. OPTIONAL FIELDS

Using `Optional[int]` lets a field be `None` when the source data doesn't include it — essential for LLM responses where a field like a reviewer's name may or may not be present.

3. TYPE COERCION

Pydantic is "smart enough" to auto-convert compatible data. If `age` is declared as `int` but the incoming value is the string `"32"`, Pydantic silently coerces it to the integer `32` instead of raising an error.

4. BUILT-IN VALIDATORS

Types like `EmailStr` come with built-in format validation — passing a malformed email (e.g. `"abc"`) immediately raises `"value is not a valid email address"`.

5. THE FIELD() FUNCTION

`Field()` lets you attach constraints, defaults, and descriptions to any attribute in one place:

```
cgpa : float = Field(gt = 0, lt = 10, default = 5,
                    description = "A decimal value representing the CGPA of the student")
```

Here `gt/lt` enforce a numeric range (rejecting a value like 12), `default` supplies a fallback value, and `description` is passed on to the LLM exactly like `Annotated` does for `TypedDict`. `Field()` can also validate against regular expressions (e.g. for phone numbers).

Converting a Pydantic Object

A Pydantic object can be converted to a plain dictionary via `dict(student)`, or to a JSON string via `student.model_dump_json()` — giving you the flexibility to work with whichever representation suits your code.

Using Pydantic with `with_structured_output()`

```
from langchain_groq import ChatGroq
from typing import TypedDict, Annotated, Optional, Literal
from pydantic import BaseModel, Field
from dotenv import load_dotenv

load_dotenv()
model = ChatGroq(model = "llama-3.3-70b-versatile")

class Review(BaseModel):
    key_themes : list[str] = Field(description = "Identify key themes discussed and write them down in list.")
    summary : str = Field(description = "Generate a brief summary of the review.")
    sentiment : Literal["pos","neg"] = Field(description = "Determine the sentiment of the review.")
    pros : Optional[list[str]] = Field(default = None, description = "List down all the pros of the product in a list.")
    cons : Optional[list[str]] = Field(default = None, description = "List down all the cons of the product in a list.")
    name : Optional[str] = Field(default = None, description = "Mention the name of the reviewer ONLY WHEN IT IS MENTIONED IN THE REVIEW. If not then return None")

structured_model = model.with_structured_output(Review)
result = structured_model.invoke("""...review text...""")
print(result)
```

Since the returned object is a Pydantic object (not a dict), its fields must be accessed with dot notation, e.g. `result.summary`, not `result["summary"]`.

COMMON MISTAKE

Trying to access a Pydantic result with dictionary-style indexing (`result["name"]`). Since `with_structured_output(PydanticModel)` returns a Pydantic object, use attribute access instead (`result.name`), or convert it to a dict first.

Quick Revision — Pydantic

- Pydantic is a data validation AND parsing library — it enforces constraints at runtime.
- `BaseModel` is the parent class every Pydantic schema inherits from.
- `Field()` adds constraints (gt/lt), default values, descriptions, and regex validation.
- Pydantic performs automatic type coercion where safely possible (e.g. "32" -> 32).
- Built-in types like `EmailStr` provide ready-made format validation.
- A Pydantic result is an object — access fields with dot notation, not dict indexing.
- Convert to dict with `dict(obj)` or to JSON with `obj.model_dump_json()`.

7. Schema Approach 3 — JSON Schema

Definition

JSON Schema is a language-agnostic way of describing the structure of a JSON object — the same document structure can be understood by Python, JavaScript, or any other language.

Why It Exists

TypedDict and Pydantic are both Python-specific. If a project spans multiple languages — for example, a Python backend and a JavaScript frontend, both of which need to agree on the same data shape — neither TypedDict nor Pydantic can be shared across that language boundary. JSON, being a universal data format, can.

Anatomy of a JSON Schema

A JSON Schema document is built from four key parts:

Key	Purpose
title	The name of the schema.
description	Optional, but useful when the schema is meant to guide an LLM.
type	The overall data type of the schema (an object corresponds to a Python dictionary).
properties	The individual attributes/fields of the schema, each with its own type and optional description.
required	The list of attribute names that must always be present — equivalent to a field not being Optional.

A Simple Example

```
{
  "title": "Student",
  "description": "Schema about Students",
  "type": "object",
  "properties": {
    "name": "string",
    "age": "integer"
  },
  "required": ["name"]
}
```

Here, `name` is mandatory (it is listed under `required`) while `age` is optional.

Using a JSON Schema with `with_structured_output()`

A more complete schema (matching the same Review use case used for TypedDict and Pydantic) mirrors the same fields, but every list-type field is expressed as `"type": "array"` with an `"items"` sub-schema, and enum-style fields use `"enum": [...]` instead of `Literal[...]`:

```
json_schema = {
    "title": "Review",
    "type": "object",
    "properties": {
        "key_themes": {
            "type": "array",
            "items": {"type": "string"},
            "description": "Identify key themes discussed and write them down in list."
        },
        "summary": {"type": "string", "description": "Generate a brief summary of the review."},
        "sentiment": {"type": "string", "enum": ["pos", "neg"], "description": "Determine the sentiment of the review."},
        "pros": {"type": ["array", "null"], "items": {"type": "string"}, "description": "List down all the pros of the product in a list."},
        "cons": {"type": ["array", "null"], "items": {"type": "string"}, "description": "List down all the cons of the product in a list."},
        "name": {"type": ["string", "null"], "description": "Mention the name of the reviewer ONLY WHEN IT IS MENTIONED. If not then return null."}
    },
    "required": ["key_themes", "summary", "sentiment"]
}

# Pass the dict directly to with_structured_output instead of a class
structured_model = model.with_structured_output(json_schema)
result = structured_model.invoke("""...review text...""")
```

The result is returned as a plain Python dictionary — just like the TypedDict version — and can be indexed with normal dict syntax.

Quick Revision — JSON Schema

- JSON Schema is a language-agnostic way to describe a JSON object's structure.
- Core keys: title, description, type, properties, required.
- Ideal when a schema must be shared across multiple languages (e.g. Python backend + JS frontend).
- Optional/nullable fields are expressed as a type array, e.g. `["array", "null"]`.
- Enum-style constraints use `"enum": [...]` instead of `Literal[...]`.
- The result returned is a plain Python dictionary, same as with TypedDict.

8. Choosing the Right Approach

Comparison

TypedDict

Only type hints — no runtime validation, no defaults, Python-only

Pydantic

Full data validation, defaults, type coercion, constraints — Python-only

JSON Schema

Universal format understood by any language — no Python-side validation

Figure 5.5 — What each schema-definition approach gives you

Approach	Runtime Validation	Defaults	Type Coercion	Cross-Language
TypedDict	No	No	No	No (Python only)
Pydantic	Yes	Yes	Yes	No (Python only)
JSON Schema	No (on the Python side)	No	No	Yes

How It Works — Decision Guide

- **Use TypedDict** when the entire project stays in Python and the schema never needs to be shared with another language — you only need type hints.
- **Use Pydantic** when you need real data validation, default values, or automatic type conversion, and the whole project is Python. This is the most commonly used option in practice.
- **Use JSON Schema** when the schema must be understood outside Python — e.g. shared with a JavaScript frontend — and you don't want to import extra Python-only libraries.

BEST PRACTICE

Since most LangChain projects live entirely within the Python ecosystem, Pydantic is generally the go-to format for defining schemas — reach for it first unless there's a specific reason (like cross-language sharing) to use JSON Schema instead.

The method Parameter

`with_structured_output()` accepts a `method` parameter that tells the model exactly how to produce structured output:

Method	When To Use
"json_mode"	When the structured output should simply be returned as JSON — the most common case. Recommended for non-OpenAI models such as Claude or Gemini, since these models natively support JSON output.
"function_calling"	When the structured output is needed because the result will be passed on to call a function/tool — typical in agent-building scenarios. This is the default and recommended method for OpenAI models.

A Known Limitation

Not every model supports structured output at all. Some open-source models lack support for both JSON mode and function calling entirely.

COMMON MISTAKE

Assuming every model can be passed to `with_structured_output()`. Certain lightweight open-source models (e.g. TinyLlama) throw an error when this function is called, because they support neither JSON mode nor function calling. For such models, Output Parsers (covered separately) must be used instead.

🔄 Quick Revision — Choosing the Right Approach

- TypedDict: pure Python, type hints only, no validation needed.
- Pydantic: pure Python, validation/defaults/coercion needed — the most common real-world choice.
- JSON Schema: schema must be shared across multiple programming languages.
- `method="json_mode"`: recommended for non-OpenAI models (Claude, Gemini).
- `method="function_calling"`: default and recommended for OpenAI models; used heavily by agents.
- Some models support neither method and cannot use `with_structured_output()` at all.

K. Key Takeaways

- `structured_output` means having an LLM return its response in a well-defined data format (typically JSON) instead of free text.
- `structured_output` stand out: data extraction into databases, building APIs on top of unstructured text, and powering agents that call tools.
- `structured_output_from_schema` is the LangChain function used for models that natively support structured output; it internally builds a hidden system prompt from your schema.
- `structured_output` gives type hints only, with no runtime validation — fine for pure-Python, low-risk projects.
- `structured_output_from_schema` adds real validation, default values, and automatic type coercion — the most commonly used schema format in practice.
- `structured_output_from_schema` is language-agnostic and the right choice when a schema must be shared outside the Python ecosystem.
- The `structured_output` parameter (`json_mode` vs. `function_calling`) controls how the model is asked to produce structured data.
- Not all models support structured output — some require Output Parsers instead, the topic of the next chapter.

R. Revision Sheet

Structured Output

An LLM response returned in a well-defined, machine-readable data format (usually JSON) rather than free text.

`with_structured_output()`

A LangChain chat-model method that accepts a schema and returns responses shaped to that schema, by auto-generating a hidden schema-describing system prompt.

TypedDict

A Python typing construct that defines expected dictionary keys/types as editor hints only — no runtime validation or defaults.

Pydantic

A data validation and parsing library; `BaseModel + Field()` enable constraints, defaults, descriptions, and automatic type coercion.

JSON Schema

A universal, language-agnostic document (title/type/properties/required) describing a JSON object's shape.

method parameter

"json_mode" for plain JSON output (non-OpenAI models); "function_calling" for tool-calling flows (default for OpenAI models).

Output Parsers

The alternative mechanism (next chapter) used when a model has no native structured-output support at all.

I. Interview Questions

Q1. What is the difference between structured and unstructured output from an LLM?

- Q2.** Why does structured output matter when integrating an LLM with other systems?
- Q3.** What does `with_structured_output()` do internally when you pass it a schema?
- Q4.** What is the key limitation of `TypedDict` compared to `Pydantic`?
- Q5.** How does `Pydantic`'s automatic type coercion behave, and when does it fail instead of coercing?
- Q6.** What is the purpose of the `Field()` function in `Pydantic`, and what can it configure?
- Q7.** When would you choose JSON Schema over `Pydantic` or `TypedDict` for defining a schema?
- Q8.** What is the difference between the `"json_mode"` and `"function_calling"` values of the `method` parameter?
- Q9.** Why might `with_structured_output()` fail entirely for certain open-source models?
- Q10.** Describe a real-world scenario where structured output would be essential for building an AI agent.

F. Further Reading

- [LangChain documentation — Structured Output \("How to return structured data from a model"\)](#)
- [Pydantic official documentation — Models, Fields, and Validators](#)
- [Python typing documentation — TypedDict](#)
- [JSON Schema official specification \(json-schema.org\)](#)
- [Hugging Face documentation — ChatHuggingFace and HuggingFaceEndpoint](#)